# Linear regression

- Linear regression is a statistical model, which is used to model the ~~regression~~ Relationship between one or more Independent variables and a dependent variable by fitting a straight line that minimizes the prediction error

- In simple linear regression
  - one dependent variable (Y) ↔ one Independent variable (x)

$$\boxed{y = mx + c}$$

- $y \rightarrow$ Dependent variable (target)
- $m \rightarrow$ slope / weight
- $x \rightarrow$ Independent variable (feature)
- $c \rightarrow$ bias / Intercept

- In multiple linear regression
  - One dependent variable (Y) ↔ more than one Independent variable ($\beta_0 \cdots \beta_n$)



Scanned with OKEN Scanner

$$\boxed{y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \varepsilon}$$

- $y \rightarrow$ Target variable / Dependent variable
- $x \rightarrow$ feature / Independent variable
- $\beta_0 \rightarrow$ bias
- $\beta_1 \cdots \beta_n \rightarrow$ weight
- $\varepsilon \rightarrow$ error term

<u>Slope (m)</u>

- The slope tells how much $y$ changes when '$x$' increased by 1 unit

$$m = \frac{\text{change in } y}{\text{change in } x}$$

<u>Intercept (c)</u>

- It tells the value of $y$ when $x = 0$

$$y = mx + b$$
$$y = m(0) + b$$
$$y = b$$



Scanned with OKEN Scanner

- In linear regression

  $y \rightarrow$ output

  $x \rightarrow$ Input

the model assumes that y changes at a constant rate with respect to x and tries to find a straight line that predicts y from x with minimal error compared to observed data

- <u>Error</u>
  - During prediction there will be a difference between actual and prediction, we measure how wrong is it

    - Actual = $y$
    - prediction = $\hat{y}$

    error = $\hat{y} - y$

    = prediction − actual

Ex:-
Actual CPU usage = 50
prediction = 45
error = −5

- we square the error because
  - It removes sign
  - penalizes error
  - It enable optimization

$$\text{error} = (\hat{y} - y)^2$$



Scanned with OKEN Scanner

Cost function / mean square error

- This function is used to measure error and turns best line into measurable objective

$$J(m,b) = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

$J$ = Cost (How bad the model is?)
$n$ = No. of Samples

- smaller $J$ = better model

Gradient desent

- Goal is to find the values of m and b that minimizes "J"

$$\Delta J = \begin{bmatrix} \frac{\partial J}{\partial m} \\ \frac{\partial J}{\partial b} \end{bmatrix}$$

- gradient is a vector of partial derivative

where $\alpha$ = learning rate
- usually used small $\alpha$ = 0.01 (or) 0.0001

$$m = m - \alpha\frac{\partial J}{\partial m}$$

$$b = b - \alpha\frac{\partial J}{\partial b}$$



Scanned with OKEN Scanner

Training or testing data

- During training
  - The model see the data
  - The model adjust the parametres to reduce cost on the data
  - The model is incentivized to perform well on those exact points

- we divide the data into 2 parts

| Dataset | |
|---|---|

↓ Training set
- used to Compute gradients
- update m, b
- minimize cost
- preferred 70% or 80% split

↓ Testing set
- only for testing the model
- model generalisation
- preferred 30% or 20% split

- If model performs poor on training and testing data then the model is underfitting

- If model performs good at train data and bad at test data is overfit



Scanned with OKEN Scanner

- If test set leak into training
  - performance metrics lie
  - production failures occurs
  - overfitting goes undetected

Mathematical Calculation of Simple linear Regression

- Hours Studied VS Exam Score

| Student | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| Hours studied | 1 | 2 | 3 | 4 | 5 | |
| Exam marks | 52 | 55 | 61 | 66 | 72 | |

- we will Compute the regression line

$$Y = b_0 + b_1 x$$

where

$b_0$ = Slope

$b_1$ = Intercept

for simple linear regression



Scanned with OKEN Scanner

$$b_1 = \frac{n\sum xY - (x)(y)}{n\sum x^2 - (\sum x)^2}$$

$$b_0 = \frac{\sum Y - b_1 \sum x}{n}$$

| X | Y | $X^2$ | XY |
|---|---|---|---|
| 1 | 52 | 1 | 52 |
| 2 | 55 | 4 | 110 |
| 3 | 61 | 9 | 183 |
| 4 | 66 | 16 | 264 |
| 5 | 72 | 25 | 360 |

$\sum x = 1+2+3+4+5 \Rightarrow 15$

$\sum Y = 52+55+61+66+72 \Rightarrow 306$

$\sum x^2 = 1+4+9+16+25 \Rightarrow 55$

$\sum xY = 52+110+183+264+360 \Rightarrow 969$

$n = 5$

$$b_1 = \frac{5*969 - 15*306}{5*55 - (15)^2}$$

$$= \frac{4845 - 4590}{275 - 225}$$

$$= \frac{255}{50}$$

$$= 5.1$$

$$b_0 = \frac{306 - 5.1*15}{5}$$

$$= \frac{306 - 76.5}{5}$$

$$= \frac{229.5}{5}$$

$$= 45.9$$

1to

• Final regression $\Rightarrow$ $y = b_0 + b_1 x$

$= 45.9 + 5.1 * x$



Scanned with OKEN Scanner

Code

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squarred_error,
                            r2_score
```

numpy/pandas : for handling arrays + tables

- train_test_split : Splits your data into train and test split
- metrics :-
  - MAE
  - MSE
  - r2_score

```
df = pd.read_csv("data.csv")
```

- read_csv is a pandas function that reads .csv files

```
feature = [["A", "B", "C", "D"]]
target = [" x"
x = df[feature]
y = df[target]
```

x is a table (It stores in 2D)

y is a Column used for target



Scanned with OKEN Scanner

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
                                                    test_size = 0.2,
                                                    random_state = 42
)
```

- train_test_split : used to divide the data into
  - training
    - X_train
    - Y_train
  - testing
    - X_test
    - Y_test

- test_size = 0.2
  - 20 % of the data is for testing
  - remaining data is for training

- random_state = 42
  - make the split reproducible (same results each run)

```
model = LinearRegression()
model.fit(X_train, Y_train)
```

- LinearRegression() → Creates the model object
- .fit(X_train, Y_train) → finds the best fit line by minimizing MSE
- under the hood it learn
  - slope (s)
  - intercept



Scanned with OKEN Scanner

Y_pred = model.predict(x_test)

- used the learned line

$$y = mx + c$$

- Returns the prediction for each input in X_test

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("mae", mae)
print("mse", mse)
print("rmse", rmse)
print("r2", r2)
```

MAE : average error in the same Unit as target

RMSE : punish big mistakes more

R2 :

0 → no better than predicting average

1 → perfect fit

r2 < 0 → worse than average



Scanned with OKEN Scanner

```
print ("Intercept :", model.intercept_)
print ("Coefficient :", model.coef_)
```

- Intercept = c
- coefficient (m) = m

  if multiple features, coef_ will have multiple slopes



Scanned with OKEN Scanner